

Objetivos y Decisiones de diseño

El objetivo del simulador es representar de manera práctica la planificación de procesos de un sistema de procesamiento de datos asociado al SIGET, permitiendo observar cómo diferentes procesos utilizan la CPU y cómo cambian de estado durante su ejecución.

Para el diseño se definieron tres procesos representativos: una emergencia de tráfico, la gestión de semáforos y el procesamiento de sensores. Cada proceso cuenta con atributos como tiempo de llegada, prioridad, tamaño de datos, tiempo de ejecución y tiempo restante.

El simulador utiliza una clase Proceso para representar la información de cada proceso y un enumerador EstadoProceso para controlar sus estados: Nuevo, Listo, En ejecución, Bloqueado y Terminado. La clase Planificador contiene la lógica necesaria para gestionar los procesos y ejecutar los algoritmos de planificación.

Algoritmos utilizados y observaciones

Algoritmo de Prioridad

El primer algoritmo implementado es planificación por prioridad. El planificador revisa los procesos que se encuentran en estado Listo y selecciona primero el que posee la prioridad más alta. En la simulación, un valor numérico menor representa una prioridad mayor. Por ejemplo, el proceso de emergencia de tráfico tiene prioridad 1, por lo que es seleccionado antes que los procesos con prioridades 2 y 3.

Round Robin

El segundo algoritmo implementado es Round Robin. Este algoritmo utiliza un quantum (**cantidad máxima de tiempo de CPU que un proceso puede utilizar durante un turno antes de que el sistema operativo le quite la CPU y se la dé a otro proceso**) de 2 unidades de tiempo. Cada proceso recibe un turno de ejecución y, si no termina durante su quantum, vuelve al estado Listo para permitir que otro proceso utilice la CPU. De esta manera se distribuye el tiempo de procesamiento entre los diferentes procesos.

Observaciones

Durante la simulación se observa que la planificación por prioridad permite atender primero los procesos considerados más importantes, por lo que resulta apropiada para situaciones donde determinados eventos requieren atención preferencial. Por otro lado, Round Robin distribuye de manera más equitativa el tiempo de CPU entre los procesos siendo adecuado para cargas donde se busca evitar que un proceso tome por completo el procesador.

Conclusiones

```
=====
SIMULADOR DE PLANIFICACION DEL SIGET
=====

Procesos          EstadoProcesos
=== ESTADO INICIAL ===
Emergencia de trafico - Estado: Nuevo
Gestion de semaforos - Estado: Nuevo
Procesamiento de sensores - Estado: Nuevo

=== PROCESOS PASAN A LISTOS ===
Emergencia de trafico - Estado: Listo planificadorRR
Gestion de semaforos - Estado: Listo
Procesamiento de sensores - Estado: Listo agregarProci

172          planificadorRR.AgregarProci
=====
ALGORITMO: PRIORIDAD planificadorRR.AgregarProci
=====

175          Console.WriteLine("=====
Ejecutando: Emergencia de trafico Console.WriteLine("
Tiempo restante: 4 Console.WriteLine("=====
Tiempo restante: 3 Console.WriteLine("=====
Tiempo restante: 2 Console.WriteLine("=====
Tiempo restante: 1 planificadorRR.PasarAListo:
Proceso terminado: Emergencia de trafico

183          Console.WriteLine();
Ejecutando: Gestion de semaforos Console.WriteLine("=== EST
Tiempo restante: 6 Console.WriteLine("=====
Tiempo restante: 5 Console.WriteLine("=====
Tiempo restante: 4 planificadorRR.MostrarProci
Tiempo restante: 3 Console.WriteLine("=====
Tiempo restante: 2 Console.WriteLine();
Tiempo restante: 1 Console.WriteLine("=== EJE
Proceso terminado: Gestion de semaforos Console.WriteLine("Quantum

188          Console.WriteLine();
Ejecutando: Procesamiento de sensores
Tiempo restante: 8 planificadorRR.EjecutarRoun
Tiempo restante: 7 Console.WriteLine("=====
Tiempo restante: 6 Console.WriteLine();
Tiempo restante: 5 Console.WriteLine("===== EST
Tiempo restante: 4 Console.WriteLine("=====
Tiempo restante: 3 planificadorRR.MostrarProci
Tiempo restante: 2 Console.WriteLine("=====
Tiempo restante: 1 Console.WriteLine();
Proceso terminado: Procesamiento de sensores Console.WriteLine();

=== ESTADO FINAL - PRIORIDAD === Console.WriteLine("=====
Emergencia de trafico - Estado: Terminado Console.WriteLine("
Gestion de semaforos - Estado: Terminado Console.WriteLine("
Procesamiento de sensores - Estado: Terminado Console.WriteLine("

Presione ENTER para continuar... ReadLine();
293          }
}
```

La simulación permitió comprobar de manera práctica cómo un planificador de procesos puede organizar la utilización de la CPU y administrar los diferentes estados de los procesos. La planificación por prioridad ofrece una ventaja para atender rápidamente procesos críticos como una emergencia de tráfico mientras que Round Robin proporciona una distribución más equitativa del tiempo de procesamiento entre las tareas.

En el contexto del SIGET, ambos algoritmos pueden ser útiles dependiendo de las características de las tareas. La prioridad resulta conveniente cuando es necesario favorecer eventos críticos mientras que Round Robin puede contribuir a gestionar de forma equilibrada procesos de procesamiento rutinario. Por lo tanto, la elección del algoritmo depende de si se busca principalmente rapidez para determinados procesos o una distribución equitativa de los recursos.